



Lecture Nine

C++ File I/O System

Ref: Herbert Schildt, Teach Yourself C++, Third Edⁿ (Chapter 9)



Using Input/Output File

A Computer File-

- is stored in a secondary storage device, i.e., hard disk;
- is permanent;
- can be used to provide input data to a program or receive output data from a program, or both;
- should reside in project directory for easy access;
- must be opened before it is used; and
- must be closed after being used.



General File I/O Steps

- Declare a file name variable;
- Associate the filename variable with disk file name;
- Open the file;
- Use the file;
- Close the file;

Stream: a sequence of character

Interactive Stream (iostream):

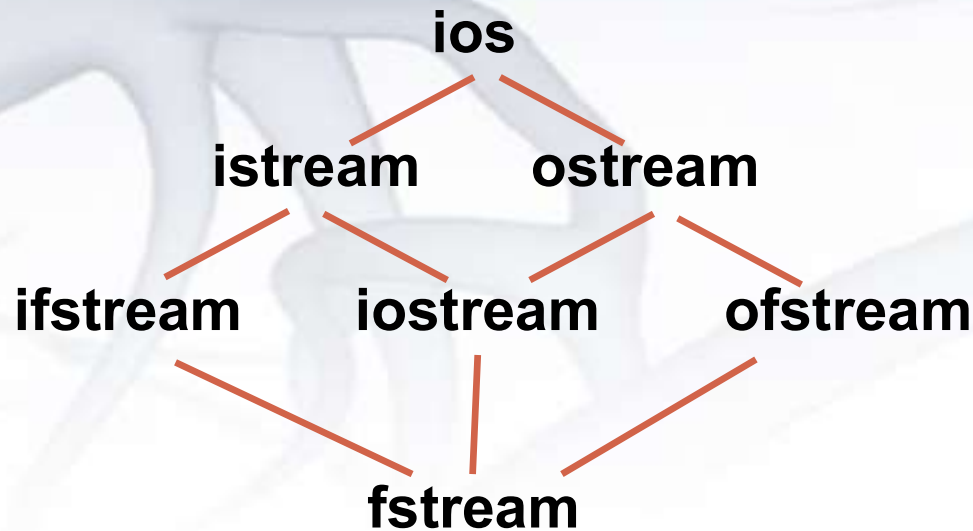
- **cin**: input stream associated with keyboard
- **cout**: output stream associated with display

File Stream (fstream):

- **ifstream**: defines new input file stream
- **ofstream**: defines new output file stream
- **fstream**: for file I/O operations
- **stringstream**: for in-memory file I/O operations



General File I/O Steps



✓ **ios** is the base class

Relations:

- ✓ **istream** and **ostream** inherit from **ios**
- ✓ **ifstream** inherits from **istream** (and **ios**)
- ✓ **ofstream** inherits from **ostream** (and **ios**)
- ✓ **iostream** inherits from **istream** and **ostream** (and **ios**)
- ✓ **fstream** inherits from **ifstream**, **iostream** and **ofstream**



A Sample File Program

```
#include <fstream>
using namespace std;

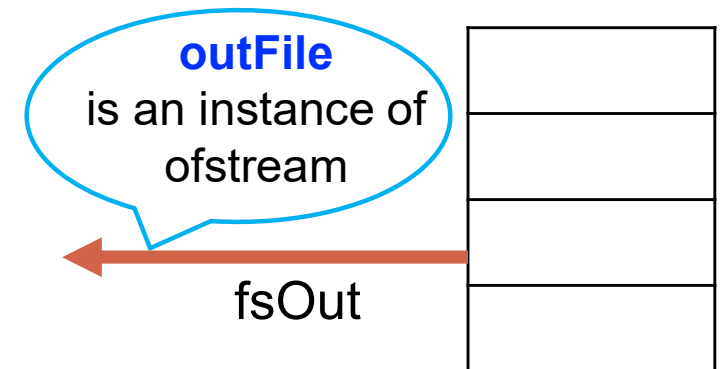
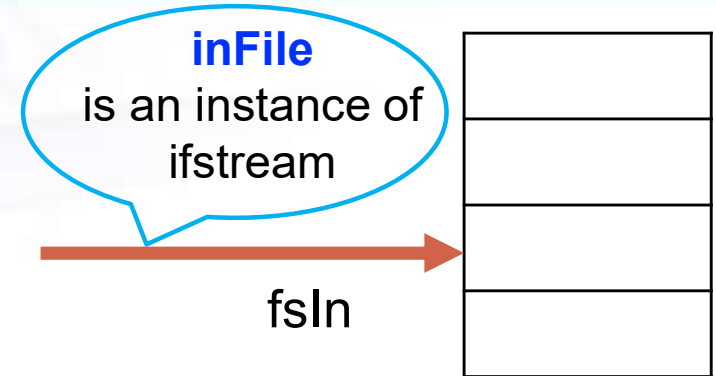
int main(){
    ifstream inFile;
    ofstream outFile;
    char ch;

    inFile.open("input.txt");
    outFile.open("output.txt");

    inFile >> ch;
    while( !inFile.eof() ){
        outFile << ch;
        inFile >> ch;
    }

    inFile.close();
    outFile.close();

    return 0;
}
```



Defining file variable and opening file in the same line:

```
ifstream inFile.open("input.txt");
ofstream outFile.open("output.txt");
```



C++ I/O operations

```
#include <fstream>
using namespace std;
```

```
int main(){
    ifstream inFile;
    ofstream outFile;
    char ch;

    inFile.open("input.txt");
    outFile.open("output.txt");

    inFile.get(ch);
    while( !inFile.eof() ){
        outFile.put(ch);
        inFile.get(ch);
    }

    inFile.close();
    outFile.close();

    return 0;
}
```

➤ Binary files access **character by character**.

istream &get(char &ch);

Reads a single character from the associated stream and put that value in *ch*.

ostream &put(char ch);

Writes *ch* to the stream and returns a reference to the stream.

istream &read(char *buf, streamsize num);

Reads *num* bytes from the invoking stream and puts them in the buffer pointed to by *buf*.

Ostream &write(const char *buf, streamsize num);

Writes *num* bytes to the associated stream from the buffer pointed to by *buf*.

Stream gcount();

Number of characters read by the last unformatted input operation

```
double num;
char str[] = "This is a test.";
outFile.write((char *) &num, sizeof(double));
outFile.write(str, strlen(str));
inFile.read((char *) &num, sizeof(double));
inFile.read(str, 14);
```



getline() function

- The prototypes of **getline()** function are as follows:

`istream &getline( char *buf, streamsize num );`

`istream &getline( char *buf, streamsize num, char delim );`

✓ The first form reads characters into the array pointed to by *buf* until either ***num - 1*** characters have been read, a **new line** is found or the **end of the file** is encountered. **New line character is extracted but it is not put into *buf*.**

✓ The second form reads characters into the array pointed to by *buf* until either ***num - 1*** characters have been read, the character specified by ***delim*** is found or the **end of the file** is encountered.

```
string line;
if(inFile.is_open())
    while(!inFile.eof()){
        getline(inFile, line);
        cout << line;
    }
```



File Open mode

➤ **openmode** is an enumeration defined in **ios**. The values of **openmode** are as follows:

ios::in	Open file to read.
ios::out	Open file to write.
ios::app	All output to be appended to the end of the file.
ios::binary	Causes a file to be opened in binary mode.
ios::trunc	Causes the contents of a pre-existing file destroyed and the file to be truncated to zero length.
ios::ate	Open with marker at the end of file.
ios::nocreate	Open only if file exists, don't create.

Examples:

```
inFile.open("input.txt", ios::in | ios::binary);  
outFile.open("output.txt", ios::app | ios::trunc);
```




More Unformatted I/O Functions

Function Prototype	Purpose
<code>int peek();</code>	Returns next character in the input stream without removing it from the stream. It returns EOF if end of file is encountered.
<code>istream &putback(char c);</code>	Returns the last character from a stream to the stream .
<code>ostream flush();</code>	✓ Information is stored in internal buffer until the buffer is full before writing them to physical device. ✓ flush() function force the information to be written into the physical device before buffer is full.

Some examples:

```
char ch = in.peek();  \\ see what type of character is next
in.putback(*p);      \\ return character to stream
*p = '\0';
```



Random Access File

- Random access to a file can be performed by **seekg()** and **seekp()** functions.
- There are two pointers associated with C++: **get pointer** and **put pointer**.
 - ❑ The **get pointer** specifies where in the file the next input operation will occur.
 - ❑ The **put pointer** specifies where in the file the next output operation will occur.



Random Access File

Function Prototype	Purpose
<code>pos_type tellg();</code>	Returns current position of get pointer. <code>pos_type</code> is an integer type defined by <code>ios</code> and capable of holding the largest value that defines a file pointer.
<code>Pos_type tellp();</code>	Returns current position of put pointer.
<code>istream &seekg(off_type offset, seekdir origin);</code>	Moves the current get pointer <i>offset</i> number of bytes from the origin. <code>off_type</code> is an integer type defined by <code>ios</code> and capable of holding the largest value that offset can be. <code>seekdir</code> is an enumeration defined by <code>ios</code> that has three values: <code>ios::beg</code> seek from beginning <code>ios::cur</code> seek from current location <code>ios::end</code> seek from end
<code>istream &seekg(pos_type position);</code>	
<code>ostream &seekp(off_type offset, seekdir origin);</code>	Moves the current put pointer <i>offset</i> number of bytes from the origin.
<code>ostream &seekp(pos_type position);</code>	



Checking File I/O Status

- I/O status is **stored** in the **error flags**.
- The following member functions are used for checking I/O status:

Function Prototype	Purpose
<code>iostate rdstate();</code>	<code>iostate</code> is an enumeration defined by <code>ios</code> that includes <code>goodbit</code> No error occurred <code>eofbit</code> end of file has been encountered <code>failbit</code> A nonfatal I/O error has occurred <code>badbit</code> A fatal I/O error has occurred
<code>bool bad();</code>	Returns true if bad bit is set.
<code>bool eof();</code>	Returns true if end of file is encountered.
<code>bool fail();</code>	Returns true if failbit is set.
<code>bool good();</code>	Returns true if there are no errors.
<code>void clear(iostate flags = ios::goodbit);</code>	If flags is goodbit (as it is by default), all error flags are cleared.



Checking File I/O Status

```
#include <iostream>
#include <fstream>
using namespace std;

void checkstatus(ifstream &in);

int main( int argc, char *argv[]){
    if ( argc != 2 ){
        cout << "Usage: Convert <input>
                <output>\n";
        return 1;
    }

    ifstream in( argv[1] );
    if ( !in ){
        cout << "Cannot open input file.\n";
        return 1;
    }

    char c;
    while(in.get(c)){
        cout << c;
        checkstatus(in);
    }
```

```
        checkstatus(in);
        in.close();

        return 0;
    }
```

```
void checkstatus ( ifstream &in){
    ios::iostate i;

    i = in.rdstate();

    if ( i & ios::eofbit) cout << "End of file has
        encountered.\n";
    else if ( i & ios::failbit) cout << "Nonfatal I/O
        error.\n";
    else if ( i & ios::badbit) cout << "Fatal I/O
        error.\n";
}
```



Customized I/O and Files

The overloading of **inserters**, **extractors** and **manipulators** used in **basic I/O** is also applicable to **file I/O**.